XCPC 个人模板

目录

A4 双栏 / GNU C++20 / 41 份模板 / 多项式代码置于末尾

保留完整源代码；行号旁 > 表示同一行的视觉折行。代码片段按依赖选用。

缺省源 2
静态 Top Tree 2
二分图最大匹配 2
Dinic 最大流 3
Primal-Dual 最小费用最大流 4
网络单纯形最小费用最大流 4
模整数与组合数 5
NTT 与多项式卷积 6
广义后缀自动机 7
AC 自动机 7
回文自动机 7
后缀数组与相邻后缀 LCP 8
后缀数组 LCP / LCS 查询 8
KMP 与 Border 8
Z 函数与扩展 KMP 9
Manacher 回文半径 9
Lyndon 分解 9
最小循环表示 9
Runs：单模哈希 9
Runs：后缀数组 10
Runs：通用查询接口 10
扩展欧几里得与模逆元 11
扩展中国剩余定理 11
Lucas 组合数 11
扩展 Lucas 组合数 11
Miller-Rabin：int 素性测试 12
Pollard-Rho：整数分解与素性测试 12
扩展 BSGS 离散对数 12
最小原根：试除 13
最小原根：Pollard-Rho 13
Cipolla 模平方根 13
类欧几里得：整除和 14
类欧几里得：整除和取模 14
万欧：非交换路径拼接 14
万欧：高度与平方和 14
杜教筛：欧拉函数前缀和 15
Min_25 筛 15
PN 筛 15
PN 筛与杜教筛 15
倒数乘法除法 16
形式幂级数与多项式 16
模整数运算 16
单位根、阶乘与逆元预处理 17
NTT：DIF 正变换 17
NTT：DIT 逆变换 17
多项式求导 17
多项式积分 17
多项式四则运算与截断 17
多项式卷积 18
多项式求逆 18
多项式指数 18
多项式对数 18
多项式平方根 18
多项式幂 18
多项式除法与余数 18
转置卷积 18
多点求值 19
二维卷积 19
二维转置卷积 19
Bostan-Mori 19
转置 Bostan-Mori 19
多项式复合逆 19
多项式复合 20
在线卷积 20
在线多项式对数 20
在线多项式指数 20
在线多项式求逆 20
阶乘组合数 20
下降幂组合数 20

缺省源

s.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define rep(i, j, k) for(int i = (j); i <= (k); i++)
#define per(i, j, k) for(int i = (j); i >= (k); i--)
#define pb emplace_back
#define fi first
#define se second
using vi = vector<int>;
using pi = pair<int, int>;

bool chmin(auto &x, const auto &y){
  if(y < x){x = y; return true;} return false;
}
bool chmax(auto &x, const auto &y){
  if(x < y){x = y; return true;} return false;
}

ostream& operator << (ostream &os, const auto &v)
  requires requires { v.begin(); } && (!requires {
v.substr(); }) {
  os << "[";
  for (int f = 0; const auto& x : v) os << (f++ ? ", " :
"") << x;
  return os << "]";
}

template<typename ...Ar> void debug(char *s, Ar... x){
  auto f = [&](auto y){
    int dep = 0;
    while(*s && !(*s == ',' && dep == 0)){
      if(*s == '(') dep++;
      if(*s == ')') dep--;
      cerr << *s++;
    }
    cerr <<" = "<< y;
    if(*s == ','){
      cerr << *s ++;
    } else{
      cerr << endl;
    }
  };

  (f(x), ...);
}
#define gdb(...) debug((char*)#__VA_ARGS__, __VA_ARGS__)

signed main(){
  #ifdef LOCAL
  assert( freopen(".in", "r", stdin) );
  // assert( freopen(".out", "w", stdout) );
  #endif
  ios::sync_with_stdio(0);
  cin.tie(0);

}
```

静态 Top Tree

static_top_tree.cpp

```cpp
// 依赖缺省源中的 vi、pi、rep、per、pb、fi；仅维护静态 Top
Tree 的结构。
// G 为 0 下标无向树，root 为原树根；非空时须 0 <= root < n。
// 原树点 0..n-1 对应叶子，内部节点 n..2n-2，rt 为 Top Tree
根。
// P 为簇父亲，L/R 为左右子簇；根父亲和叶子儿子均为 -1。
// type：叶子 -1，rake 0（保留左簇下界），compress
1（左上、右下）。
// 固定原树拓扑，不支持 link/cut；build 会清空并重建结构。
// 空树 rt = -1；要求 n <= INT_MAX / 2 且内存足够。
// 此实现构建 O(n log n)、空间 O(n)，结构高度 O(log n)。
struct Tree{
  int n = 0, rt = -1;
  vi P, L, R, type;
  Tree(){}
  Tree(vector<vi> G, int root = 0){
    build(move(G), root);
  }
  int opt(int u, int v, int op){
    int x = int(size(P));
    P.pb(-1);
    L.pb(u);
    R.pb(v);
    type.pb(op);
    P[u] = P[v] = x;
    return x;
  }
  // a 中的权重为所含原树点数，仅用于平衡，顺序不可改变。
  pi mg(vector<pi> a, int op){
    if(size(a) == 1){
      return a[0];
    }
    int sum = 0;
    for(auto [u, s]:a){
      sum += s;
    }
    vector<pi> le, ri;
    for(auto [u, s]:a){
      (sum > s ? le : ri).pb(u, s);
      sum -= s * 2;
    }
    auto [u, su] = mg(move(le), op);
    auto [v, sv] = mg(move(ri), op);
    return {opt(u, v, op), su + sv};
  }
  void build(vector<vi> G, int root = 0){
    n = int(size(G)), rt = -1;
    P.assign(n, -1);
    L.assign(n, -1);
    R.assign(n, -1);
    type.assign(n, -1);
    if(!n){
      return;
    }
    int m = n * 2 - 1;
    P.reserve(m);
    L.reserve(m);
    R.reserve(m);
    type.reserve(m);
    vi ord{root}, pa(n, -1), sz(n, 1);
    rep(i, 0, n - 1){
      int u = ord[i];
      if(pa[u] != -1){
        G[u].erase(find(G[u].begin(), G[u].end(),
pa[u]));
      }
      for(int v:G[u]){
        pa[v] = u;
        ord.pb(v);
      }
    }
    per(i, n - 1, 0){
      int u = ord[i];
      for(int v:G[u]){
        sz[u] += sz[v];
      }
      if(!G[u].empty()){
        iter_swap(G[u].begin(),
max_element(G[u].begin(), G[u].end(), [&](int x, int y){
          return sz[x] < sz[y];
        }));
      }
    }
    auto Rake = [&](auto &r, auto &c, int u)->pi {
      vector<pi> a{{G[u][0], 1}};
      rep(i, 1, int(size(G[u])) - 1){
        a.pb(c(r, c, G[u][i]));
      }
      return mg(move(a), 0);
    };
    auto Compress = [&](auto &r, auto &c, int u)->pi {
      vector<pi> a{{u, 1}};
      while(!G[u].empty()){
        a.pb(r(r, c, u));
        u = G[u][0];
      }
      return mg(move(a), 1);
    };
    rt = Compress(Rake, Compress, root).fi;
  }
};
```

二分图最大匹配

bipartite_matching.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define rep(i, j, k) for(int i = (j); i <= (k); i++)
#define per(i, j, k) for(int i = (j); i >= (k); i--)
#define pb emplace_back
#define fi first
#define se second
using vi = vector<int>;
using pi = pair<int, int>;

bool chmin(auto &x, const auto &y){
  if(y < x){x = y; return true;} return false;
}
```

```cpp
bool chmax(auto &x, const auto &y){
  if(x < y){x = y; return true;} return false;
}

ostream& operator << (ostream &os, const auto &v)
  requires requires { v.begin(); } && (!requires {
v.substr(); }) {
  os << "[";
  for (int f = 0; const auto& x : v) os << (f++ ? ", " :
"") << x;
  return os << "]";
}

template<typename ...Ar> void debug(char *s, Ar... x){
  auto f = [&](auto y){
    int dep = 0;
    while(*s && !(*s == ',' && dep == 0)){
      if(*s == '(') dep++;
      if(*s == ')') dep--;
      cerr << *s++;
    }
    cerr <<" = "<< y;
    if(*s == ','){
      cerr << *s ++;
    } else{
      cerr << endl;
    }
  };

  (f(x), ...);
}
#define gdb(...) debug((char*)#__VA_ARGS__, __VA_ARGS__)

pair<int, vi> bipartite_matching(int n, int m, const
vector<vi> &G){
  vi vis(n), dis(n), match(n, -1), pre(m, -1);

  auto bfs = [&]()->bool {
    fill(dis.begin(), dis.end(), -1);

    queue<int> Q;
    rep(i, 0, n - 1){
      if(match[i] == -1){
        dis[i] = 0;
        Q.emplace(i);
      }
    }
    bool res = 0;
    while(Q.size()){
      int u = Q.front();
      Q.pop();
            vis[u] = 0;
      for(int v:G[u]){
        if(pre[v] == -1){
          res = 1;
        } else if(dis[ pre[v] ] == -1){
          dis[ pre[v] ] = dis[u] + 1;
          Q.emplace(pre[v]);
        }
      }
    }
    return res;
  };

  auto dfs = [&](auto &self, int u)->int {
    if(vis[u]){
      return 0;
    }
    vis[u] = 1;
    for(int v:G[u]){
      if(pre[v] == -1 || (dis[ pre[v] ] == dis[u] + 1 &&
self(self, pre[v]))){
        pre[v] = u, match[u] = v;
        return 1;
      }
    }
    return 0;
  };

  int res = 0;
  while(bfs()){
    rep(i, 0, n - 1){
      if(dis[i] == 0){
        res += dfs(dfs, i);
      }
    }
  }
  return {res, match};
}

```

Dinic 最大流

dinic.cpp

```cpp
namespace _flow{
  template<typename Cap> struct flow{
    static constexpr Cap inf =
numeric_limits<Cap>::max();
    struct edge{
      int v = -1;
      Cap w = 0;
      int nxt = 0;
    };
    vector<edge> E;

    int n, S, T, tot;
    vector<int> bg, cur, dis, Q;
    int head, tail;

    void init(){
      n = 0, S = 0, T = 0, tot = 2, head = 0, tail = 0;
      E.resize(2);
    }
    flow(){
      init();
    }
    flow(int _n){
      init();
      n = _n;
      bg.resize(n);
      cur.resize(n);
      dis.resize(n, -1);
      Q.resize(n + 1);
    }

    int link(int u, int v, Cap w){
      E.pb(edge{v, w, bg[u]}); bg[u] = tot++;
      E.pb(edge{u, 0, bg[v]}); bg[v] = tot++;
      return tot - 2;
    }
    bool bfs(){
      head = 1;
      while(tail){
        int u = Q[tail--];
        dis[u] = -1, cur[u] = bg[u];
      }
      dis[S] = 0;
      Q[++tail] = S;
      while(head <= tail){
        int u = Q[head++];
        for(int id = bg[u]; id; id = E[id].nxt){
          int v = E[id].v;
          Cap w = E[id].w;
          if(w && dis[v] == -1){
            dis[v] = dis[u] + 1;
            Q[++tail] = v;
            if(v == T){
              return 1;
            }
          }
        }
      }
      return 0;
    }
    Cap dfs(int u, Cap rst){
      if(u == T){
        return rst;
      }
      Cap pre = rst;
      for(; cur[u]; cur[u] = E[cur[u]].nxt){
        int v = E[cur[u]].v;
        Cap w = E[cur[u]].w;
        if(w && dis[u] + 1 == dis[v]){
          Cap f = dfs(v, min(rst, w));
          E[cur[u]].w -= f, E[cur[u] ^ 1].w += f, rst -=
f;
          if(!rst){
            break;
          }
        }
      }
      return pre - rst;
    }
    Cap solve(){
      rep(i, 0, n - 1){
        dis[i] = -1, cur[i] = bg[i];
      }
      Cap res = 0;
      while(bfs()){
        res += dfs(S, inf);
      }
      return res;
```

```
    }
    Cap solve(int _S, int _T){
      S = _S, T = _T;
      return solve();
    }
  };
}
using namespace _flow;
```

Primal-Dual 最小费用最大流

primal_dual.cpp

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define rep(i, j, k) for(int i = (j); i <= (k); i++)
#define per(i, j, k) for(int i = (j); i >= (k); i--)
#define pb emplace_back
#define fi first
#define se second
using vi = vector<int>;
using pi = pair<int, int>;

bool chmin(auto &x, const auto &y){
  if(y < x){x = y; return true;} return false;
}
bool chmax(auto &x, const auto &y){
  if(x < y){x = y; return true;} return false;
}

ostream& operator << (ostream &os, const auto &v)
  requires requires { v.begin(); } && (!requires {
v.substr(); }) {
  os << "[";
  for (int f = 0; const auto& x : v) os << (f++ ? ", " :
"") << x;
  return os << "]";
}

template<typename ...Ar> void debug(char *s, Ar... x){
  auto f = [&](auto y){
    int dep = 0;
    while(*s && !(*s == ',' && dep == 0)){
      if(*s == '(') dep++;
      if(*s == ')') dep--;
      cerr << *s++;
    }
    cerr <<" = "<< y;
    if(*s == ','){
      cerr << *s ++;
    } else{
      cerr << endl;
    }
  };

  (f(x), ...);
}
#define gdb(...) debug((char*)#__VA_ARGS__, __VA_ARGS__)

namespace _flow{
  const i64 inf_cap = 1e18;
  const i64 inf_cost = 1e18;
  template<typename Cap = i64, typename Cost = i64>
struct mcmf{
    struct edge{
      int v;
      Cap w;
      Cost c;
      int r;
    };
    int n, m, S, T;
    vi cur;
    vector<Cost> h, dis;
    vector<bool> vis;
    vector< vector<edge> > G;

    mcmf(int _n, int _s, int _t){
      n = _n, S = _s, T = _t;
      cur.resize(n);
      h.resize(n);
      dis.resize(n);
      vis.resize(n);
      G.resize(n);
    }
    mcmf(int _n = 0){
      mcmf(_n + 2, _n, _n + 1);
    }
    void link(int u, int v, Cap w, Cost c){
      G[u].pb(edge{v, w, c, int(G[v].size())});
      G[v].pb(edge{u, 0, -c, int(G[u].size()) - 1});
    }
    void spfa(){
      ranges::fill(h, inf_cost);
      h[S]=0, vis[S]=1;
      queue<int> Q;
      Q.emplace(S);
      while(!Q.empty()){
        int u=Q.front();
        Q.pop();
        vis[u] = 0;
        for(auto &[v, w, c, r]:G[u]){
          if(w && chmin(h[v], h[u] + c) && !vis[v]){
            Q.emplace(v);
            vis[v] = 1;
          }
        }
      }
    }
    bool dij(){
      ranges::fill(dis, inf_cost);
      dis[S] = 0;
      priority_queue< pair<Cost, int>, vector<
pair<Cost, int> >, greater<> > Q;
      Q.emplace(0, S);
      while(size(Q)){
        auto [d, u] = Q.top();
        Q.pop();
        if(dis[u] != d){
          continue;
        }
        for(auto &[v, w, c0, r]:G[u]){
          Cost c = c0 + (h[u] - h[v]);
          if(w && dis[u] + c <dis[v]){
            dis[v] = dis[u] + c;
            Q.emplace(dis[v], v);
          }
        }
      }
      return dis[T] != inf_cost;
    }
    Cap dfs(int u, Cap rst){
      if(u == S){
        return rst;
      }
      vis[u] = 1;
      Cap pre = rst;
      for(; cur[u] < ssize(G[u]); cur[u] ++){
        auto &[v, w, c0, r] = G[u][cur[u]];
        Cost c = -c0 + (h[v] - h[u]);
        if(dis[v] + c == dis[u] && G[v][r].w && !vis[v]){
          Cap f = dfs(v, min(rst, G[v][r].w));
          G[u][cur[u]].w += f, G[v][r].w -= f, rst -= f;
          if(!rst){
            break;
          }
        }
      }
      vis[u] = 0;
      return pre - rst;
    }
    pair<Cap, Cost> dinic(){
      spfa();
      Cap flow = 0;
      Cost cost = 0;
      while(1){
        rep(i, 0, n - 1){
          h[i] += dis[i];
        }
        dij();
        if(dis[T] == inf_cost){
          break;
        }
        ranges::fill(cur, 0);
        Cap f = dfs(T, inf_cap);
        flow += f;
        cost += f * (h[T] + dis[T]);
      }
      return {flow, cost};
    }
  };
}
using _flow::mcmf;

```

网络单纯形最小费用最大流

network_simplex.cpp

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define rep(i, j, k) for(int i = (j); i <= (k); i++)
#define per(i, j, k) for(int i = (j); i >= (k); i--)
```

```cpp
#define pb emplace_back
#define fi first
#define se second
using vi = vector<int>;
using pi = pair<int, int>;

template<typename T0, typename T1> bool chmin(T0 &x, const T1 &y){
  if(y < x){x = y; return true;} return false;
}
template<typename T0, typename T1> bool chmax(T0 &x, const T1 &y){
  if(x < y){x = y; return true;} return false;
}

template<typename T> void debug(char *s, T x){
  cerr << s <<" = "<< x <<endl;
}
template<typename T, typename ...Ar> void debug(char *s, T x, Ar... y){
  int dep = 0;
  while(!(*s == ',' && dep == 0)){
    if(*s == '(') dep++;
    if(*s == ')') dep--;
    cerr << *s++;
  }
  cerr <<" = "<< x <<",";
  debug(s + 1, y...);
}
#define gdb(...) debug((char*)#__VA_ARGS__, __VA_ARGS__)

namespace _flow{
  // maxcost: 单位流量带来的最大费用变化
  // inf: 总流量和费用的最大绝对值
  constexpr i64 maxcap = 1e18;
  constexpr i64 maxcost = 1e9, inf = 1e18;

  template<typename Cap, typename Cost> struct flow{
    struct edge{
      int v = -1;
      Cap w = 0;
      Cost c = 0;
      int nxt = -1;
    };
    vector<edge> e;

    int n, s, t, tm;
    vector<int> bg, vis, fa, fae, cyc;
    Cost cost;
    vector<Cost> pi;

    void init(){
      n = 0, s = 0, t = 0, tm = 0, cost = 0;
      e.pb();
      e.pb();
    }
    flow(){
      init();
    }
    flow(int _n){
      init();
      n = _n;
      bg.resize(n, -1);
      vis.resize(n);
      fa.resize(n, -1);
      fae.resize(n);
      pi.resize(n);
    }

    int link(int u, int v, Cap w, Cost c){
      e.pb(v, w, c, bg[u]); bg[u] = int(size(e)) - 1;
      e.pb(u, 0, -c, bg[v]); bg[v] = int(size(e)) - 1;
      return int(size(e)) - 2;
    }
    void dfs(int u){
      vis[u] = 1;
      for(int i = bg[u]; ~i; i = e[i].nxt){
        int v = e[i].v;
        if(!vis[v] && e[i].w){
          fa[v] = u, fae[v] = i;
          dfs(v);
        }
      }
    }
    Cost phi(int u){
      if(u < 0){
        return 0;
      }
      if(vis[u] == tm){
        return pi[u];
      }
      vis[u] = tm;
      return pi[u] = phi(fa[u]) + e[fae[u]].c;
    }
    void pushflow(int id){
      tm ++;
      int p = e[id ^ 1].v, q = e[id].v;
      while(~p){
        vis[p] = tm, p = fa[p];
      }
      while(~q && vis[q] != tm){
        vis[q] = tm, q = fa[q];
      }
      int lca = q;

      int mnv = -1, op = 2;
      Cap mnf = e[id].w;
      for(int i = e[id].v; i != lca; i = fa[i]){
        cyc.pb(fae[i] ^ 1);
        if(e[fae[i] ^ 1].w < mnf){
          mnv = i, mnf = e[fae[i] ^ 1].w, op = 0;
        }
      }
      for(int i = e[id ^ 1].v; i != lca; i = fa[i]){
        cyc.pb(fae[i]);
        if(e[fae[i]].w < mnf){
          mnv = i, mnf = e[fae[i]].w, op = 1;
        }
      }
      cyc.pb(id);

      for(int eid:cyc){
        e[eid].w -= mnf, e[eid ^ 1].w += mnf, cost += mnf * e[eid].c;
      }
      cyc.clear();

      if(op == 2){
        return;
      }
      int lste = id ^ (op ^ 1), lst = e[lste].v, now = e[lste ^ 1].v;
      while(lst != mnv){
        vis[now] = 0, lste ^= 1;
        swap(lste, fae[now]);
        swap(lst, fa[now]);
        swap(lst, now);
      }
    }
    pair<Cap, Cost> solve(){
      dfs(t);
      vis[t] = tm = 2, fa[t] = -1, cost = 0;

      int cnt = int(size(e));
      for(int i = 0, pre = cnt - 1; i != pre; i = (i == cnt - 1)? 0: i+1){
        if(e[i].w && e[i].c + (phi(e[i ^ 1].v) - phi(e[i].v)) < 0){
          pre = i;
          pushflow(i);
        }
      }
      return {e.back().w, cost};
    }
    pair<Cap, Cost> solve(int _s, int _t){
      s = _s, t = _t;
      link(t, s, inf, -maxcost);

      auto [w, c] = solve();
      pair<Cap, Cost> res{w, c + e.back().w * maxcost};
      e.pop_back();
      e.pop_back();
      return res;
    }
  };
}
using namespace _flow;

```

模整数与组合数

mint.cpp

```cpp

using u32 = uint32_t;
using u64 = uint64_t;
constexpr int mod = 998244353;
struct mint{
  u32 x;

  mint(): x(0){}
  mint(int _x){
    _x %= mod;
    if(_x < 0) _x += mod;
```

```
    x = _x;
  }

  mint qpow(int y = mod - 2)const {
    assert(y >= 0);
    mint t = *this, res = 1;
    while(y){
      if(y % 2) res *= t;
      t *= t;
      y /= 2;
    }
    return res;
  }

  mint& operator += (const mint &B){
    if((x += B.x) >= mod) x -= mod;
    return *this;
  }
  mint& operator -= (const mint &B){
    if((x -= B.x) >= mod) x += mod;
    return *this;
  }
  mint& operator *= (const mint &B){
    x = u32((u64)x * B.x % mod);
    return *this;
  }
  mint& operator /= (const mint &B){
    return *this *= B.qpow();
  }
  friend mint operator + (mint A, const mint &B){
    return mint(A) += B;
  }
  friend mint operator - (mint A, const mint &B){
    return mint(A) -= B;
  }
  friend mint operator * (mint A, const mint &B){
    return mint(A) *= B;
  }
  friend mint operator / (mint A, const mint &B){
    return mint(A) /= B;
  }
  mint operator - ()const {
    return mint() - *this;
  }

  friend ostream& operator << (ostream &os, const mint
 > &obj){
    os << obj.x;
    return os;
  }
};

const int N = 1e6 + 5;
mint fac[N], ifac[N];

auto INIT = [](){
  fac[0] = 1;
  rep(i, 1, N - 1){
    fac[i] = fac[i - 1] * i;
  }
  ifac[N - 1] = 1 / fac[N - 1];
  per(i, N - 1, 1){
    ifac[i - 1] = ifac[i] * i;
  }
  return true;
}();

mint C(int m, int n){
  return n < 0 || n > m? mint(): fac[m] * ifac[n] *
 > ifac[m - n];
}
```

NTT 与多项式卷积

ntt.cpp

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i, j, k) for(int i = (j); i <= (k); i++)

using u32 = uint32_t;
using u64 = uint64_t;
constexpr int mod = 998244353;

struct mint{
  u32 x;

  mint(): x(0){}
  mint(int _x){
    _x %= mod;
    if(_x < 0) _x += mod;
    x = _x;
  }

  mint qpow(int y = mod - 2)const {
    assert(y >= 0);
    mint t = *this, res = 1;
    while(y){
      if(y % 2) res *= t;
      t *= t;
      y /= 2;
    }
    return res;
  }

  mint& operator += (const mint &B){
    if((x += B.x) >= mod) x -= mod;
    return *this;
  }
  mint& operator -= (const mint &B){
    if((x -= B.x) >= mod) x += mod;
    return *this;
  }
  mint& operator *= (const mint &B){
    x = u32(u64(x) * B.x % mod);
    return *this;
  }
  mint& operator /= (const mint &B){
    return *this *= B.qpow();
  }
  friend mint operator + (mint A, const mint &B){
    return mint(A) += B;
  }
  friend mint operator - (mint A, const mint &B){
    return mint(A) -= B;
  }
  friend mint operator * (mint A, const mint &B){
    return mint(A) *= B;
  }
  friend mint operator / (mint A, const mint &B){
    return mint(A) /= B;
  }
};

constexpr int LG = 21, S = 1 << LG;
vector<mint> P(S), ip2(LG + 1);

auto INIT = [](){
  rep(i, 0, LG - 1){
    mint stp = mint(3).qpow(mod >> (i + 1));
    P[1 << i] = 1;
    rep(j, (1 << i) + 1, (2 << i) - 1){
      P[j] = P[j - 1] * stp;
    }
  }
  ip2[0] = 1;
  rep(i, 1, LG) ip2[i] = ip2[i - 1] / 2;
  return true;
}();

struct poly : vector<mint>{
  using vector<mint>::vector;

  void DIF(int lg){
    for(int i = 1 << (lg - 1); i; i /= 2){
      for(int j = 0; j < (1 << lg); j += 2 * i){
        rep(k, 0, i - 1){
          mint A = (*this)[j + k], B = (*this)[j + k +
 > i];
          (*this)[j + k] = A + B;
          (*this)[j + k + i] = (A - B) * P[i + k];
        }
      }
    }
  }

  void DIT(int lg){
    for(int i = 1; i < (1 << lg); i *= 2){
      for(int j = 0; j < (1 << lg); j += 2 * i){
        rep(k, 0, i - 1){
          mint A = (*this)[j + k], B = (*this)[j + k +
 > i] * P[i + k];
          (*this)[j + k] = A + B;
          (*this)[j + k + i] = A - B;
        }
      }
    }
    reverse(begin() + 1, end());
  }

  friend poly conv(const poly &_A, const poly &_B){
    if(min(ranges::size(_A), ranges::size(_B)) <= 64){
      poly res(ranges::size(_A) + ranges::size(_B) - 1);
      rep(i, 0, int(ranges::size(_A)) - 1){
```

```
        rep(j, 0, int(ranges::size(_B)) - 1){
          res[i + j] += _A[i] * _B[j];
        }
      }
      return res;
    }

    poly A = _A, B = _B;
    int len = int(ranges::size(A) + ranges::size(B) - 1);
    int lg = __lg(len * 2 - 1);
    A.resize(1 << lg), B.resize(1 << lg);
    A.DIF(lg), B.DIF(lg);
    rep(i, 0, (1 << lg) - 1) A[i] *= B[i];
    A.DIT(lg), A.resize(len);
    for(auto &x:A) x *= ip2[lg];
    return A;
  }
};
```

广义后缀自动机

string/gsam.cpp

```
// 依赖缺省源中的 i64、vi、rep、pb；字符编号 [0, S)，根为 1，0 表示空转移。
// n 为插入字符串的总长度上界（含重复前缀），须非负且 2n + 2 不溢出 int。
// 预留容量，转移表按需初始化；每个新串从 p = 1 开始，逐字符执行 p = ins(p, c)。
// 固定字符集时构建 O(n)，空间 O(Sn)；solve 返回不同非空子串数、状态数（含根）。
struct SAM{
  static const int S = 26;
  vector<array<int, S>> s;
  vi fa, len;
  int tot = 1;
  SAM(int n){
    int m = n * 2 + 2;
    s.reserve(m);
    s.resize(2);
    fa.resize(m);
    len.resize(m);
  }
  // 内部辅助函数，要求 s[p][c] != 0。
  int split(int p, int c){
    int v = s[p][c];
    if(len[v] == len[p] + 1){
      return v;
    }
    int q = ++tot;
    s.pb(s[v]);
    fa[q] = fa[v], len[q] = len[p] + 1;
    fa[v] = q;
    for(; p && s[p][c] == v; p = fa[p]){
      s[p][c] = q;
    }
    return q;
  }
  int ins(int p, int c){
    if(s[p][c]){
      return split(p, c);
    }
    int u = ++tot;
    s.pb();
    len[u] = len[p] + 1;
    for(; p && !s[p][c]; p = fa[p]){
      s[p][c] = u;
    }
    fa[u] = (p ? split(p, c) : 1);
    return u;
  }
  pair<i64, int> solve()const {
    i64 res = 0;
    rep(i, 2, tot){
      res += len[i] - len[fa[i]];
    }
    return {res, tot};
  }
};
```

AC 自动机

string/ac.cpp

```
// 依赖缺省源中的 vi、rep、pb；根为 1，字符编号 [0, S)。
// n 为模式串总长上界（非串数），须非负且 n + 2 不溢出 int。
// 每个模式串从 p = 1 插入，p = ins(p, c)；保存终点，重复串终点相同。
// 全部插入后仅调用一次 build，此后不可插入或再次 build。
// 构建后 s 为完整转移表，fa[1] = 0 为哨兵。
// 模式串非空，文本可空；固定字符集时插入 O(总长)，构建 O(S * tot)，空间 O(Sn)。
struct AC{
  static const int S = 26;
  vector<array<int, S>> s;
  vi fa;
  int tot = 1;
  AC(int n){
    int m = n + 2;
    s.reserve(m);
    s.resize(2);
    fa.resize(m);
  }
  int ins(int p, int c){
    if(!s[p][c]){
      s[p][c] = ++tot;
      s.pb();
    }
    return s[p][c];
  }
  void build(){
    s[0].fill(1);
    queue<int> q;
    q.emplace(1);
    while(!q.empty()){
      int u = q.front();
      q.pop();
      rep(c, 0, S - 1){
        int &v = s[u][c];
        if(v){
          fa[v] = s[fa[u]][c];
          q.emplace(v);
        } else{
          v = s[fa[u]][c];
        }
      }
    }
  }
};
```

回文自动机

string/pam.cpp

```
// 依赖缺省源中的 vi、pb；str 为小写字母串，位置从 0 开始。
// 1 为偶根（len = 0），2 为奇根（len = -1），fa[1] = fa[2] = 2。
// 两根 dep = 0；非根 dep[u] = dep[fa[u]] + 1，为非空回文后缀个数。
// n 为处理长度上界，须非负且 n + 3 不溢出 int；最多 n + 2 个节点。
// 按 pos = 0, 1, ... 调用 u = ins(u, pos)，初始 u = 1；返回最长回文后缀节点。
// str 可预先赋值或逐字追加，已处理的前缀不可修改；固定字符集下 O(n) 时间、O(Sn) 空间。
struct PAM{
  static const int S = 26;
  string str;
  vector<array<int, S>> s;
  vi fa, len, dep;
  int tot = 2;
  PAM(int n){
    int m = n + 3;
    str.reserve(n);
    s.reserve(m);
    s.resize(3);
    fa.resize(m);
    len.resize(m);
    dep.resize(m);
    fa[1] = fa[2] = 2;
    len[1] = 0, len[2] = -1;
  }
  int getfa(int u, int pos){
    while(pos <= len[u] || str[pos] != str[pos - len[u] - 1]){
      u = fa[u];
    }
    return u;
  }
  int ins(int u, int pos){
    int c = str[pos] - 'a';
    u = getfa(u, pos);
    if(!s[u][c]){
      s[u][c] = ++tot;
      s.pb();
      len[tot] = len[u] + 2;
      fa[tot] = (u == 2 ? 1 : s[getfa(fa[u], pos)][c]);
      dep[tot] = dep[fa[tot]] + 1;
    }
    return s[u][c];
  }
};
```

后缀数组与相邻后缀 LCP

string/sa.cpp

```cpp
// 依赖缺省源中的 vi、rep、per；按 ASCII（0..127）排序，支持空串。
// sa[r] 为排名 r 的后缀起点，rk[i] 为后缀 i 的排名，均从 0 开始。
// 返回 {rk, sa}；倍增构建 O(n log n) 时间、O(n) 空间。
// 长度须能用 int 表示；按字节处理，不进行 Unicode 字符解码。
pair<vi, vi> SA(const string &s){
  int n = int(size(s));
  vi rk(n), sa(n);
  if(!n){
    return {};
  }
  vi cnt(max(n, 128)), tmp(n), nrk(n);
  for(char c:s){
    cnt[c]++;
  }
  rep(i, 1, 127){
    cnt[i] += cnt[i - 1];
  }
  per(i, n - 1, 0){
    sa[--cnt[s[i]]] = i;
  }
  int ccnt = 0;
  rep(i, 0, n - 1){
    if(i && s[sa[i]] != s[sa[i - 1]]){
      ccnt++;
    }
    rk[sa[i]] = ccnt;
  }
  ccnt++;
  for(int len = 1; ccnt < n; len *= 2){
    fill(cnt.begin(), cnt.begin() + ccnt, 0);
    rep(i, 0, n - 1){
      cnt[rk[i]]++;
    }
    rep(i, 1, ccnt - 1){
      cnt[i] += cnt[i - 1];
    }
    int cur = 0;
    rep(i, n - len, n - 1){
      tmp[cur++] = i;
    }
    rep(i, 0, n - 1){
      if(sa[i] >= len){
        tmp[cur++] = sa[i] - len;
      }
    }
    per(i, n - 1, 0){
      sa[--cnt[rk[tmp[i]]]] = tmp[i];
    }
    ccnt = 0;
    nrk[sa[0]] = 0;
    rep(i, 1, n - 1){
      int x = sa[i - 1], y = sa[i];
      int a = (x < n - len ? rk[x + len] : -1);
      int b = (y < n - len ? rk[y + len] : -1);
      if(rk[x] != rk[y] || a != b){
        ccnt++;
      }
      nrk[y] = ccnt;
    }
    ccnt++;
    rk.swap(nrk);
    if(ccnt == n){
      break;
    }
  }
  return {move(rk), move(sa)};
}

// sa、rk 须为同一字符串的 SA 结果；O(n) 时间、O(n) 返回空间。
// height[0] = 0，height[r] = lcp(s[sa[r - 1]..], s[sa[r]..])。
vi get_height(const string &s, const vi &sa, const vi &rk){
  int n = int(size(s));
  vi height(n);
  int k = 0;
  rep(i, 0, n - 1){
    if(!rk[i]){
      k = 0;
      continue;
    }
    if(k){
      k--;
    }
    int j = sa[rk[i] - 1];
    while(k < n - i && k < n - j && s[i + k] == s[j + k]){
      k++;
    }
    height[rk[i]] = k;
  }
  return height;
}
```

后缀数组 LCP / LCS 查询

string/lcp_sa.cpp

```cpp
// 依赖缺省源及 sa.cpp；按 ASCII（0..127）比较，不保留输入串引用。
// 查询下标 [0, n)，lcs(x, y) 包含位置 x、y；空串可构造，不可查询。
// 预处理 O(n log n) 时间、空间；lcp/lcs/cmp 均为 O(1)。
// 要求长度能用 int 表示；两份 ST 在 n = 10^6 时约占 145 MiB。
struct sa_lcp{
  struct rmq{
    vector<vi> f;
    rmq(vi a = {}){
      int n = int(size(a));
      if(!n){
        return;
      }
      f.pb(move(a));
      rep(k, 1, __lg(n)){
        int len = 1 << k;
        f.pb(vi(n - len + 1));
        rep(i, 0, n - len){
          f[k][i] = min(f[k - 1][i], f[k - 1][i + len / 2]);
        }
      }
    }
    int get(int l, int r)const {
      int k = __lg(r - l + 1);
      return min(f[k][l], f[k][r - (1 << k) + 1]);
    }
  };
  int n;
  vi rk, rrk;
  rmq pre, suf;
  sa_lcp(const string &s): n(int(size(s))){
    auto [a, b] = SA(s);
    pre = rmq(get_height(s, b, a));
    rk = move(a);
    string t(s.rbegin(), s.rend());
    auto [c, d] = SA(t);
    suf = rmq(get_height(t, d, c));
    rrk = move(c);
  }
  int lcp(int x, int y)const {
    if(x == y){
      return n - x;
    }
    int l = rk[x], r = rk[y];
    if(l > r){
      swap(l, r);
    }
    return pre.get(l + 1, r);
  }
  int lcs(int x, int y)const {
    if(x == y){
      return x + 1;
    }
    int l = rrk[n - 1 - x], r = rrk[n - 1 - y];
    if(l > r){
      swap(l, r);
    }
    return suf.get(l + 1, r);
  }
  bool cmp(int x, int y)const {
    return rk[x] < rk[y];
  }
};
```

KMP 与 Border

string/kmp.cpp

```cpp
// 依赖缺省源中的 vi、rep；按 ASCII（0..127）比较，长度须能用 int 表示。
// fail[i] 为 s[0..i] 最长真 border 的末下标，无 border 为 -1；长度为 fail[i] + 1。
// 空串返回空数组；O(n) 时间、O(n) 空间。
vi kmp(const string &s){
  int n = int(size(s));
  vi fail(n, -1);
  rep(i, 1, n - 1){
    int p = fail[i - 1];
    while(p != -1 && s[i] != s[p + 1]){
```

```
      p = fail[p];
    }
    if(s[i] == s[p + 1]){
      p++;
    }
    fail[i] = p;
  }
  return fail;
}
```

Z 函数与扩展 KMP

string/z.cpp

```
// 依赖缺省源中的 vi、rep；按 ASCII（0..127）比较，长度须能用 int 表示。
// z[i] = lcp(s, s[i..])，z[0] = n；空串返回空数组。
// O(n) 时间、O(n) 空间；用于扩展匹配时，初始区间应为空（r = -1）。
vi zfunc(const string &s){
  int n = int(size(s));
  vi z(n);
  if(n){
    z[0] = n;
  }
  int l = 0, r = -1;
  rep(i, 1, n - 1){
    if(i <= r){
      z[i] = min(z[i - l], r - i + 1);
    }
    while(z[i] < n - i && s[z[i]] == s[i + z[i]]){
      z[i]++;
    }
    if(i + z[i] - 1 > r){
      l = i, r = i + z[i] - 1;
    }
  }
  return z;
}

// p[i] = lcp(t, s[i..])；s 为文本串，t 为模式串。
// 空文本返回空数组，空模式返回全 0；O(n + m) 时间、空间。
vi exkmp(const string &s, const string &t){
  int n = int(size(s)), m = int(size(t));
  vi p(n);
  if(!n || !m){
    return p;
  }
  vi z = zfunc(t);
  int l = 0, r = -1;
  rep(i, 0, n - 1){
    if(i <= r){
      p[i] = min(z[i - l], r - i + 1);
    }
    while(p[i] < m && p[i] < n - i && s[i + p[i]] == t[p[i]]){
      p[i]++;
    }
    if(i + p[i] - 1 > r){
      l = i, r = i + p[i] - 1;
    }
  }
  return p;
}

// 示例：exkmp("aaaabaa", "aaaaa") 返回 {4, 3, 2, 1, 0, 2, 1}。
```

Manacher 回文半径

string/manacher.cpp

```
// 依赖缺省源中的 vi、rep；变换为 ?s[0]?s[1]?...?，长度为 2 * |s| + 1。
// d[i] 为不含中心的半径，覆盖变换串 [i - d[i], i + d[i]]。
// 对应原串 [(i - d[i]) / 2, (i + d[i]) / 2)，最长回文长度为 max(d)。
// 空串返回 {0}；按 ASCII（0..127）比较，要求 |s| <= (INT_MAX - 1) / 2；O(n) 时间、空间。
vi manacher(const string &s){
  int m = int(size(s)), n = m * 2 + 1;
  string t(n, '?');
  rep(i, 0, m - 1){
    t[i * 2 + 1] = s[i];
  }
  vi d(n);
  int l = 0, r = -1;
  rep(i, 0, n - 1){
    if(i <= r){
      d[i] = min(d[l + (r - i)], r - i);
    }
    while(d[i] < i && d[i] < n - i - 1 && t[i - d[i] - 1] == t[i + d[i] + 1]){
      d[i]++;
    }
    if(i + d[i] > r){
      l = i - d[i], r = i + d[i];
    }
  }
  return d;
}
```

Lyndon 分解

string/lyndon.cpp

```
// 依赖缺省源中的 vi、pb；按 ASCII（0..127）进行 Lyndon 分解。
// 返回边界 {0, ..., n}，各段为 [res[i], res[i + 1])；空串返回 {0}。
// 各段均为 Lyndon 串，且字典序不增；长度须能用 int 表示。
// Duval 算法 O(n) 时间、O(1) 辅助空间，返回数组空间 O(分段数 + 1)。
vi lyndon(const string &s){
  int n = int(size(s)), i = 0;
  vi res{0};
  while(i < n){
    int j = i, k = i + 1;
    while(k < n && s[j] <= s[k]){
      j = (s[j] == s[k] ? j + 1 : i);
      k++;
    }
    int len = k - j;
    while(i <= j){
      i += len;
      res.pb(i);
    }
  }
  return res;
}
```

最小循环表示

string/min_rotation.cpp

```
// 依赖缺省源中的 vi；返回字典序最小循环表示的起始下标（从 0 开始）。
// 多个起点对应相同最小序列时返回最小下标；空序列返回 -1。
// 支持任意 int 元素；长度须不超过 INT_MAX / 2。
// Duval 算法 O(n) 时间、O(1) 辅助空间，不修改输入。
int min_rotation(const vi &a){
  int n = int(size(a)), i = 0, ans = -1;
  while(i < n){
    ans = i;
    int j = i, k = i + 1;
    while(k < n * 2){
      int x = a[(j < n ? j : j - n)];
      int y = a[(k < n ? k : k - n)];
      if(x > y){
        break;
      }
      j = (x == y ? j + 1 : i);
      k++;
    }
    int len = k - j;
    while(i <= j){
      i += len;
    }
  }
  return ans;
}
```

Runs：单模哈希

string/runs_hash.cpp

```
// 依赖缺省源中的 i64、vi、rep、per、pb；与 runs_sa.cpp 二选一。
// 返回排序、去重后的 {l, r, p}，0 下标闭区间，p 为最小周期。
// 按 ASCII（0..127）比较，支持空串；要求 n < INT_MAX。
// O(n log n) 时间、O(n) 空间；固定单模哈希存在碰撞反例，不保证正确。
vector<tuple<int, int, int>> get_runs(const string &s){
  int n = int(size(s));
  if(n < 2){
    return {};
  }
  const int base = 107897, mod = 998244353;
  vi pw(n + 1), pre(n + 1);
  pw[0] = 1;
  rep(i, 0, n - 1){
    pw[i + 1] = int(i64(pw[i]) * base % mod);
    pre[i + 1] = int((i64(pre[i]) * base + s[i]) % mod);
  }
  auto val = [&](int l, int r){
    int res = int(pre[r] - i64(pre[l]) * pw[r - l] % mod);
```

```
    return (res < 0 ? res + mod : res);
  };
  auto lcp = [&](int x, int y){
    if(s[x] != s[y]){
      return 0;
    }
    int l = 0, r = n - max(x, y);
    while(l < r){
      int mid = l + (r - l + 1) / 2;
      if(val(x, x + mid) == val(y, y + mid)){
        l = mid;
      } else{
        r = mid - 1;
      }
    }
    return l;
  };
  auto lcs = [&](int x, int y){
    if(s[x] != s[y]){
      return 0;
    }
    int l = 0, r = min(x, y) + 1;
    while(l < r){
      int mid = l + (r - l + 1) / 2;
      if(val(x - mid + 1, x + 1) == val(y - mid + 1, y + 1)){
        l = mid;
      } else{
        r = mid - 1;
      }
    }
    return l;
  };
  auto cmp = [&](int x, int y){
    int len = lcp(x, y);
    if(x + len == n || y + len == n){
      return x + len == n && y + len != n;
    }
    return s[x + len] < s[y + len];
  };
  vector<tuple<int, int, int>> res;
  auto chk = [&](int x, int y){
    if(s[x] != s[y]){
      return;
    }
    int ri = lcp(x, y), le = lcs(x, y);
    if(le + ri - 1 >= y - x){
      auto cur = tuple{x - le + 1, y + ri - 1, y - x};
      if(res.empty() || res.back() != cur){
        res.pb(cur);
      }
    }
  };
  vi stk;
  rep(rev, 0, 1){
    stk.clear();
    per(i, n - 1, 0){
      while(!stk.empty() && (rev ? cmp(stk.back(), i) : cmp(i, stk.back()))){
        stk.pop_back();
      }
      if(!stk.empty()){
        chk(i, stk.back());
      }
      stk.pb(i);
    }
  }
  sort(res.begin(), res.end());
  res.erase(unique(res.begin(), res.end()), res.end());
  return res;
}
```

Runs：后缀数组

string/runs_sa.cpp

```
// 接在缺省源与 sa.cpp 后使用；与 runs_hash.cpp 二选一。
// 返回排序、去重后的 {l, r, p}，0 下标闭区间，p 为最小周期。
// 按 ASCII（0..127）比较，支持空串；长度须能用 int 表示。
// O(n log n) 时间、空间；两份 ST 在 n = 10^6 时合计约 145 MiB。
vector<tuple<int, int, int>> get_runs(const string &s){
  int n = int(size(s));
  if(n < 2){
    return {};
  }
  auto build = [&](const string &t){
    auto [rk, sa] = SA(t);
    vector<vi> f;
    f.pb(get_height(t, sa, rk));
    rep(k, 1, __lg(n)){
      int len = 1 << k;
      f.pb(vi(n - len + 1));
      rep(i, 0, n - len){
        f[k][i] = min(f[k - 1][i], f[k - 1][i + len / 2]);
      }
    }
    return pair{move(rk), move(f)};
  };
  auto [rk, f] = build(s);
  auto [rrk, g] = build(string(s.rbegin(), s.rend()));
  // l、r 为不同后缀的排名，查询 height[min(l,r) + 1 .. max(l,r)]。
  auto query = [&](const vector<vi> &f, int l, int r){
    if(l > r){
      swap(l, r);
    }
    int k = __lg(r - l);
    return min(f[k][l + 1], f[k][r - (1 << k) + 1]);
  };
  auto lcp = [&](int x, int y){
    return (x == y ? n - x : query(f, rk[x], rk[y]));
  };
  auto lcs = [&](int x, int y){
    return (x == y ? x + 1 : query(g, rrk[n - 1 - x], rrk[n - 1 - y]));
  };
  auto cmp = [&](int x, int y){
    return rk[x] < rk[y];
  };
  vector<tuple<int, int, int>> res;
  auto chk = [&](int x, int y){
    if(s[x] != s[y]){
      return;
    }
    int ri = lcp(x, y), le = lcs(x, y);
    if(le + ri - 1 >= y - x){
      auto cur = tuple{x - le + 1, y + ri - 1, y - x};
      if(res.empty() || res.back() != cur){
        res.pb(cur);
      }
    }
  };
  vi stk;
  rep(rev, 0, 1){
    stk.clear();
    per(i, n - 1, 0){
      while(!stk.empty() && (rev ? cmp(stk.back(), i) : cmp(i, stk.back()))){
        stk.pop_back();
      }
      if(!stk.empty()){
        chk(i, stk.back());
      }
      stk.pb(i);
    }
  }
  sort(res.begin(), res.end());
  res.erase(unique(res.begin(), res.end()), res.end());
  return res;
}
```

Runs：通用查询接口

string/runs.cpp

```
// 依赖缺省源、sa.cpp、lcp_sa.cpp。
// get_runs<sa_lcp>(s)，也可传入已构造的查询对象。
// 返回排序、去重后的 {l, r, p}，0 下标闭区间，p 为最小周期；空串返回空。
// LCP 对象须对应同一 s，提供精确的 LCP、LCS 和后缀比较。
// 共 O(n) 次查询；单次查询 O(Q) 时枚举含排序 O(nQ + n log n)，辅助空间 O(n)。
// 另计查询结构预处理与空间；使用 sa_lcp 时总时间为 O(n log n)。
vector<tuple<int, int, int>> get_runs(const string &s, const auto &t){
  int n = int(size(s));
  vector<tuple<int, int, int>> res;
  auto chk = [&](int x, int y){
    if(s[x] != s[y]){
      return;
    }
    int ri = t.lcp(x, y), le = t.lcs(x, y);
    if(le + ri - 1 >= y - x){
      auto cur = tuple{x - le + 1, y + ri - 1, y - x};
      if(res.empty() || res.back() != cur){
        res.pb(cur);
      }
    }
  };
  vi stk;
  rep(rev, 0, 1){
    stk.clear();
```

```
    per(i, n - 1, 0){
      while(!stk.empty() && (rev ? t.cmp(stk.back(), i)
: t.cmp(i, stk.back()))){
        stk.pop_back();
      }
      if(!stk.empty()){
        chk(i, stk.back());
      }
      stk.pb(i);
    }
  }
  sort(res.begin(), res.end());
  res.erase(unique(res.begin(), res.end()), res.end());
  return res;
}

template<typename T> auto get_runs(const string &s){
  return get_runs(s, T(s));
}
```

扩展欧几里得与模逆元

number_theory/exgcd.cpp

```
// T 为 int 或 i64，a,b 非负；返回 g=gcd(a,b)，使
a*x+b*y=g；x、y 须为不同变量。
// (0,0) 返回 g=0、x=1、y=0；O(log(max(a,b)+2))
时间与递归栈空间。
template<class T>
T exgcd(T a, T b, T &x, T &y){
  if(b == 0){
    x = 1, y = 0;
    return a;
  }
  T g = exgcd(b, a % b, y, x);
  y -= a / b * x;
  return g;
}

// 0 <= x <= INT_MAX，1 <= p <= INT_MAX，且
gcd(x,p)=1；返回 [0,p) 内的逆元。
// 不检查可逆性；系数已在 (-p,p)，只需修正负数；p=1 返回 0。
// 时间与递归栈空间均为 O(log(p+1))。
int inv(int x, int p){
  int res, _;
  exgcd(x, p, res, _);
  if(res < 0){
    res += p;
  }
  return res;
}

// x 为任意 int，1 <= p <= INT_MAX；返回 [0,p)
内的逆元，无逆元返回 -1。
// p=1 返回 0；模数不必为质数。时间与递归栈空间均为
O(log(p+1))。
int safe_inv(int x, int p){
  x %= p;
  if(x < 0){
    x += p;
  }
  int res, y;
  if(exgcd(x, p, res, y) != 1){
    return -1;
  }
  if(res < 0){
    res += p;
  }
  return res;
}
```

扩展中国剩余定理

number_theory/excrt.cpp

```
// 依赖 exgcd.cpp 中的 exgcd，以及缺省源中的 i64、fi、se。
using i128 = __int128;

// {模数,余数}；模数为正，余数可为任意 i64；lcm 须不超过
LLONG_MAX。
// 返回 {lcm,最小非负解}，无解为 {-1,-1}；无解标记可继续合并。
// O(log(min(A.fi,B.fi)+1)) 时间与递归栈空间；不修改输入。
pair<i64, i64> crt_merge(const pair<i64, i64> &A, const
pair<i64, i64> &B){
  if(A.fi == -1 || B.fi == -1){
    return {-1, -1};
  }
  auto [a, r] = A;
  auto [b, s] = B;
  r %= a, s %= b;
  if(r < 0){
    r += a;
  }
  if(s < 0){
    s += b;
  }
  i64 x, y, g = exgcd(a, b, x, y), d = s - r;
  if(d % g){
    return {-1, -1};
  }
  i64 m = a / g * b;
  x = i64((i128(x) * (d / g) * a + r) % m);
  if(x < 0){
    x += m;
  }
  return {m, x};
}
```

Lucas 组合数

number_theory/lucas.cpp

```
// 依赖缺省源中的 i64、vi、rep、per；p 须为质数，2 <= p <=
INT_MAX。
// 构造预处理 [0,p) 的阶乘与逆阶乘，O(p) 时间、O(p)
空间，须有足够内存。
struct Lucas{
  int p;
  vi fac, ifac;
  Lucas(int p): p(p), fac(p), ifac(p){
    fac[0] = 1;
    rep(i, 1, p - 1){
      fac[i] = int(i64(fac[i - 1]) * i % p);
    }
    ifac[p - 1] = fac[p - 1]; // Wilson：(p-1)! ≡
-1，其逆元也是 -1。
    per(i, p - 1, 1){
      ifac[i - 1] = int(i64(ifac[i]) * i % p);
    }
  }
  // 0 <= m < p；n<0 或 n>m 返回 0，其余返回 C(m,n) mod
p；O(1)。
  int C(int m, int n)const {
    if(n < 0 || n > m){
      return 0;
    }
    return int(i64(fac[m]) * ifac[n] % p * ifac[m - n] %
p);
  }
  // 0 <= m <= LLONG_MAX；n<0 或 n>m 返回 0，C(0,0)=1。
  // 时间与递归栈空间均为 O(1+log_p(n+1))，最多递归 64 层。
  int solve(i64 m, i64 n)const {
    if(n < 0 || n > m){
      return 0;
    }
    if(n == 0){
      return 1;
    }
    return int(i64(C(int(m % p), int(n % p))) * solve(m
/ p, n / p) % p);
  }
};
```

扩展 Lucas 组合数

number_theory/exlucas.cpp

```
// 依赖 exgcd.cpp 中的 inv，以及缺省源中的 i64、vi、rep。
// C(m,n) mod mod；m>=0，mod>=1；n<0 或 n>m 返回 0，mod=1
返回 0。
// 模数可合数；O(mod + log(mod+1)*log(m+2))
时间，O(最大质数幂因子) 空间。
int exlucas(i64 m, i64 n, int mod){
  if(n < 0 || n > m || mod == 1){
    return 0;
  }
  i64 res = 0;
  for(int p = 2, t = mod; t > 1;){
    if(t % p){
      p++;
      continue;
    }
    int q = 1;
    while(t % p == 0){
      q *= p;
      t /= p;
    }
    vi f(q, 1);
    rep(i, 1, q - 1){
      f[i] = (i % p ? int(i64(f[i - 1]) * i % q) : f[i -
1]);
    }
    auto fac = [&](i64 x){
      i64 w = 1, c = 0;
      while(x){
        if(x / q % 2){
```

```
          w = w * f[q - 1] % q;
        }
        w = w * f[int(x % q)] % q;
        x /= p;
        c += x;
      }
      return pair{w, c}; // 去除全部 p 因子的阶乘、v_p(x!)。
    };
    auto [a, ca] = fac(m);
    auto [b, cb] = fac(n);
    auto [c, cc] = fac(m - n);
    i64 r = a * inv(int(b * c % q), q) % q;
    for(i64 e = ca - cb - cc; e && r; e--){
      r = r * p % q;
    }
    i64 k = mod / q;
    res = (res + k * inv(int(k % q), q) * r) % mod;
  }
  return int(res);
}
```

Miller-Rabin：int 素性测试

number_theory/miller_rabin_int.cpp

```
// 依赖缺省源中的 i64、rep；适用于完整有符号 int 范围，x<2 返回 false。
// 固定底数 2,7,61；O(log(x+1)) 时间、O(1) 空间，无随机误判。
namespace PR{
  // 0 <= x < mod，y>=0，mod>=2。
  int qpow(int x, int y, int mod){
    int res = 1;
    while(y){
      if(y % 2){
        res = int(i64(res) * x % mod);
      }
      x = int(i64(x) * x % mod);
      y /= 2;
    }
    return res;
  }
  bool isprime(int x){
    if(x < 2 || x % 2 == 0){
      return x == 2;
    }
    const int T[]{2, 7, 61};
    int t = __builtin_ctz(x - 1);
    auto chk = [&](int a){
      a %= x;
      if(a == 0){
        return true;
      }
      int r = qpow(a, (x - 1) >> t, x);
      if(r == 1 || r == x - 1){
        return true;
      }
      rep(i, 1, t - 1){
        r = int(i64(r) * r % x);
        if(r == x - 1){
          return true;
        }
      }
      return false;
    };
    for(int a:T){
      if(!chk(a)){
        return false;
      }
    }
    return true;
  }
}
```

Pollard-Rho：整数分解与素性测试

number_theory/pollard_rho.cpp

```
// 内含 i64 Miller–Rabin；仅依赖缺省源中的 i64、pb、rep 和标准库。
namespace PR{
  using i128 = __int128;
  // 0 <= x < mod，y>=0，mod>=2。
  i64 qpow(i64 x, i64 y, i64 mod){
    i64 res = 1;
    while(y){
      if(y % 2){
        res = i64(i128(res) * x % mod);
      }
      x = i64(i128(x) * x % mod);
      y /= 2;
    }
    return res;
  }
  // 完整有符号 i64 范围的确定性判定，x<2 返回 false。
  bool isprime(i64 x){
    if(x < 2 || x % 2 == 0){
      return x == 2;
    }
    const int T[]{2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    // const int T[]{2, 7, 61}; // 仅判定 int 范围时，用此行替换上行。
    int t = __builtin_ctzll(x - 1);
    auto chk = [&](i64 a){
      a %= x;
      if(a == 0){
        return true;
      }
      i64 r = qpow(a, (x - 1) >> t, x);
      if(r == 1 || r == x - 1){
        return true;
      }
      rep(i, 1, t - 1){
        r = i64(i128(r) * r % x);
        if(r == x - 1){
          return true;
        }
      }
      return false;
    };
    for(int a:T){
      if(!chk(a)){
        return false;
      }
    }
    return true;
  }
  mt19937_64 rnd(3663);
  // x 须为合数；返回严格介于 1 和 x 之间的因子，失败重新随机尝试。
  i64 rho(i64 x){
    if(x % 2 == 0){
      return 2;
    }
    for(;;){
      i64 c = i64(rnd() % x), p = 0, q = 0, prd = 1;
      for(int goal = 1; goal <= (1 << 20); goal *= 2){
        rep(i, 1, goal){
          q = i64((i128(q) * q + c) % x);
          prd = i64(i128(prd) * abs(q - p) % x);
          if(i % 127 == 0 && gcd(prd, x) != 1){
            break;
          }
        }
        i64 g = gcd(prd, x);
        if(g != 1){
          if(g != x){
            return g;
          }
          break;
        }
        p = q;
      }
    }
  }
  void work(i64 x, vector<i64> &a){
    if(x == 1){
      return;
    }
    if(isprime(x)){
      a.pb(x);
      return;
    }
    i64 g = rho(x);
    work(g, a);
    work(x / g, a);
  }
  // 1 <= x <= LLONG_MAX；返回不降质因子数组，保留重复项，x=1 返回空数组。
  vector<i64> solve(i64 x){
    vector<i64> a;
    work(x, a);
    sort(a.begin(), a.end());
    return a;
  }
}
```

扩展 BSGS 离散对数

number_theory/bsgs.cpp

```
// 依赖缺省源中的 i64、rep、se；0 <= a,b <= INT_MAX，1 <= p <= INT_MAX。
// 返回 a^x = b (mod p) 的最小非负解，无解返回 -1；约定 0^0=1，p=1 返回 0。
```

```cpp
// 不要求互质或质数模数；哈希平均 O(1) 时，时间 O(sqrt(p)+log(p))，空间 O(sqrt(p))。
int bsgs(int a, int p, int b){
  a %= p, b %= p;
  int t = __lg(p), z = 1 % p;
  rep(i, 0, t){
    if(z == b){
      return i;
    }
    if(i < t){
      z = int(i64(z) * a % p);
    }
  }
  int g = gcd(z, p);
  if(b % g){
    return -1;
  }
  z /= g, b /= g, p /= g; // z*a^y = b (mod p)，此时 gcd(a,p)=1。
  int B = int(sqrt(p)) + 1;
  unordered_map<int, int> vis;
  vis.reserve(B);
  for(int j = 0, r = b; j < B; j++){
    vis[r] = j; // 相同余数保留最大 j，使 i*B-j 最小。
    r = int(i64(r) * a % p);
  }
  int step = 1, v = a % p;
  for(int e = B; e; e /= 2){
    if(e % 2){
      step = int(i64(step) * v % p);
    }
    v = int(i64(v) * v % p);
  }
  rep(i, 1, B){
    z = int(i64(z) * step % p);
    auto it = vis.find(z);
    if(it != vis.end()){
      return int(i64(i) * B - it->se + t);
    }
  }
  return -1;
}
```

最小原根：试除

number_theory/primitive_root.cpp

```cpp
// 试除版，仅依赖缺省源中的 i64、pb 和标准库；与 Pollard–Rho 版二选一。
// 1<=d<=2^62-1；返回最小原根，不存在（含 d<=1）返回 -1；大素数可能很慢。
i64 primitive_root(i64 d){
  using i128 = __int128;
  if(d == 2 || d == 4){
    return d - 1;
  }
  if(d <= 1 || d % 4 == 0){
    return -1;
  }
  i64 n = (d % 2 ? d : d / 2), p = 3;
  while(p * p <= n && n % p != 0){
    p += 2;
  }
  if(p * p > n){
    p = n;
  }
  i64 t = n;
  while(t % p == 0){
    t /= p;
  }
  if(t != 1){
    return -1;
  }
  i64 phi = n / p * (p - 1), v = p - 1;
  vector<i64> f;
  for(i64 q = 2; q * q <= v; q++){
    if(v % q == 0){
      f.pb(q);
      while(v % q == 0){
        v /= q;
      }
    }
  }
  if(v > 1){
    f.pb(v);
  }
  if(n != p){
    f.pb(p); // phi=p^(k-1)*(p-1)，仅需分解 p-1。
  }
  auto qpow = [](i64 x, i64 y, i64 mod){
    i64 res = 1;
    while(y){
      if(y % 2){
        res = i64(i128(res) * x % mod);
      }
      x = i64(i128(x) * x % mod);
      y /= 2;
    }
    return res;
  };
  for(i64 g = 2; g < d; g++){
    if(gcd(g, d) != 1){
      continue;
    }
    bool ok = true;
    for(i64 q:f){
      if(qpow(g, phi / q, d) == 1){
        ok = false;
        break;
      }
    }
    if(ok){
      return g;
    }
  }
  return -1;
}
```

最小原根：Pollard-Rho

number_theory/primitive_root_pollard_rho.cpp

```cpp
// Pollard–Rho 版，先粘贴 pollard_rho.cpp；与试除版二选一。
// 返回 d 的最小原根，不存在（含 d<=1）返回 -1。
// 支持完整正 i64 范围；仅 2、4、p^k、2p^k 有原根，p 为奇素数、k>=1。
i64 primitive_root(i64 d){
  if(d == 2 || d == 4){
    return d - 1;
  }
  if(d <= 1 || d % 4 == 0){
    return -1;
  }
  i64 n = (d % 2 ? d : d / 2), p = n;
  while(!PR::isprime(p)){
    p = PR::rho(p);
  }
  i64 t = n;
  while(t % p == 0){
    t /= p;
  }
  if(t != 1){
    return -1;
  }
  i64 phi = n / p * (p - 1);
  auto f = PR::solve(p - 1);
  if(n != p){
    f.pb(p); // phi=p^(k-1)*(p-1)，已知的 p 幂不必再分解。
  }
  f.erase(unique(f.begin(), f.end()), f.end());
  for(i64 g = 2; g < d; g++){
    if(gcd(g, d) != 1){
      continue;
    }
    bool ok = true;
    for(i64 q:f){
      if(PR::qpow(g, phi / q, d) == 1){
        ok = false;
        break;
      }
    }
    if(ok){
      return g;
    }
  }
  return -1;
}
```

Cipolla 模平方根

number_theory/mod_sqrt.cpp

```cpp
// Cipolla；仅依赖缺省源中的 i64、fi、se 和标准库。
// p 须为质数，2<=p<=2^63-1，n 为任意 i64；返回最小非负平方根，无解返回 -1。
// 理想随机模型下期望 O(log p) 时间，O(1) 空间；不检查素性。
i64 mod_sqrt(i64 n, i64 p){
  using i128 = __int128;
  n %= p;
  if(n < 0){
    n += p;
  }
  if(p == 2 || n == 0){
    return n;
  }
```

```
  auto qpow = [&](i64 x, i64 y){
    i64 res = 1;
    while(y){
      if(y % 2){
        res = i64(i128(res) * x % p);
      }
      x = i64(i128(x) * x % p);
      y /= 2;
    }
    return res;
  };
  if(qpow(n, p / 2) != 1){
    return -1;
  }
  if(p % 4 == 3){
    i64 r = qpow(n, p / 4 + 1);
    return min(r, p - r);
  }
  mt19937_64 rnd(3663);
  i64 a, w;
  do{
    a = i64(rnd() % p);
    w = i64((i128(a) * a - n + p) % p);
  } while(qpow(w, p / 2) != p - 1);
  auto mul = [&](pair<i64, i64> x, pair<i64, i64> y){
    i64 t = i64(i128(x.se) * y.se % p); //
三因子连乘须先取模。
    return pair{i64((i128(x.fi) * y.fi + i128(t) * w) %
p),
      i64((i128(x.fi) * y.se + i128(x.se) * y.fi) % p)};
  };
  pair<i64, i64> r{1, 0}, x{a, 1}; // (u,v) 表示
u+v*sqrt(w)。
  for(i64 e = p / 2 + 1; e; e /= 2){
    if(e % 2){
      r = mul(r, x);
    }
    x = mul(x, x);
  }
  return min(r.fi, p - r.fi);
}
```

类欧几里得：整除和

number_theory/floor_sum.cpp

```
// F=sum_{i=0}^n floor((a*i+b)/c)，n<0 返回
0；仅依赖缺省源的 i64。
// 0<=a,b,n<=2^62-1，1<=c<=2^62-1；答案须<=2^63-1。
// O(1+log(c+1)) 时间，O(1) 空间；闭区间参数顺序为 n,a,b,c。
i64 floor_sum(i64 n, i64 a, i64 b, i64 c){
  using i128 = __int128;
  if(n < 0){
    return 0;
  }
  n++;
  i64 res = 0;
  for(;;){
    res += i64(i128(n) * (n - 1) / 2 * (a / c) + i128(n)
* (b / c));
    a %= c, b %= c;
    i128 y = i128(a) * n + b;
    if(y < c){
      return res;
    }
    n = i64(y / c), b = i64(y % c);
    swap(a, c);
  }
}
```

类欧几里得：整除和取模

number_theory/floor_sum_mod.cpp

```
// 同 floor_sum 的闭区间 [0,n] 与参数顺序，答案固定对
998244353 取模。
// n<0 返回 0；0<=a,b,n<=2^62-1，1<=c<=2^62-1；仅依赖 i64。
// O(1+log(c+1)) 时间，O(1) 空间；不要求 c 可逆或为质数。
int floor_sum_mod(i64 n, i64 a, i64 b, i64 c){
  using i128 = __int128;
  const i64 mod = 998244353, inv2 = 499122177;
  if(n < 0){
    return 0;
  }
  n++;
  i64 res = 0;
  for(;;){
    i64 t = n % mod, s = t * ((n - 1) % mod) % mod *
inv2 % mod;
    res = (res + s * (a / c % mod) + t * (b / c % mod))
% mod;
    a %= c, b %= c;
    i128 y = i128(a) * n + b;
    if(y < c){
      return int(res);
    }
    n = i64(y / c), b = i64(y % c);
    swap(a, c);
  }
}
```

万欧：非交换路径拼接

number_theory/euclid.cpp

```
// k>=0；T{} 须为拼接单位元，operator+
须结合且表示从左到右拼接，不要求交换。
template<class T>
T monoid_pow(T x, i64 k){
  T res{};
  while(k){
    if(k % 2){
      res = res + x;
    }
    k /= 2;
    if(k){
      x = x + x;
    }
  }
  return res;
}

// 万欧；0<=p,l<=2^62-1，1<=q<=2^62-1，0<=r<q；仅依赖 i64。
// 返回 i=1..l 依次拼接 U^(floor((p*i+r)/q)-floor((p*(i-1)+
r)/q)) R。
// O(log(p+q+1)+log(l+1)) 次拼接；初始高度的 U^(b/q)
由调用方处理。
template<class T>
T euclid(i64 p, i64 q, i64 r, i64 l, T U, T R){
  using i128 = __int128;
  if(l == 0){
    return {};
  }
  if(p >= q){
    return euclid(p % q, q, r, l, U, monoid_pow(U, p /
q) + R);
  }
  i64 m = i64((i128(p) * l + r) / q);
  if(m == 0){
    return monoid_pow(R, l);
  }
  i64 head = (q - r - 1) / p, tail = l - i64((i128(q) *
m - r - 1) / p);
  return monoid_pow(R, head) + U
    + euclid(q, p, (q - r - 1) % p, m - 1, R, U) +
monoid_pow(R, tail);
}
```

万欧：高度与平方和

number_theory/floor_moments.cpp

```
// 先粘贴 euclid.cpp；统计每个 R 处的高度及其平方，R 的下标从 0
开始。
struct FloorMoments{
  static const int mod = 998244353;
  int r = 0, u = 0, sum = 0, sq = 0, mix = 0;
  friend FloorMoments operator+(const FloorMoments &A,
const FloorMoments &B){
    i64 r = A.r, u = A.u, tri = i64(B.r) * (B.r - 1) / 2
% mod;
    return {(A.r + B.r) % mod, (A.u + B.u) % mod,
      int((A.sum + B.sum + u * B.r) % mod),
      int((A.sq + B.sq + 2 * u * B.sum + B.r * u % mod *
u) % mod),
      int((A.mix + B.mix + B.r * u % mod * r + u * tri +
r * B.sum) % mod)};
  }
  // 返回 {sum y_i, sum y_i^2, sum
i*y_i}，i=0..n，y_i=floor((a*i+b)/c)。
  // n<0 返回全零；0<=a,b,n<=2^62-1，1<=c<=2^62-1；结果均模
mod。
  static array<int, 3> solve(i64 n, i64 a, i64 b, i64 c){
    if(n < 0){
      return {};
    }
    FloorMoments U{0, 1}, R{1};
    auto z = FloorMoments{0, int(b / c % mod)} + R +
euclid(a, c, b % c, n, U, R);
    return {z.sum, z.sq, z.mix};
  }
};
```

杜教筛：欧拉函数前缀和

number_theory/dujiao.cpp

```cpp
// 杜教筛示例：solve(n) 返回 sum_{i=1}^n phi(i)。
// 0<=n<=INT_MAX；预筛 [1,m]，要求 1<=m<INT_MAX 且 floor(sqrt(n))<=m。
// 一般 h=f*g，有 g(1)F(n)=H(n)-sum_{i=2}^n g(i)F(n/i)，F,H 为前缀和。
// 此处 phi*1=id；平方根拆分双曲线两侧，再补回重复的正方形。
struct Dujiao{
  vi phi;
  vector<i64> sphi;
  unordered_map<int, i64> mem;
  Dujiao(int m = 1000000): phi(m + 1), sphi(m + 1){
    vi pr;
    phi[1] = 1;
    rep(i, 2, m){
      if(!phi[i]){
        pr.pb(i);
        phi[i] = i - 1;
      }
      for(int p:pr){
        if(p > m / i){
          break;
        }
        int j = i * p;
        if(i % p == 0){
          phi[j] = phi[i] * p;
          break;
        }
        phi[j] = phi[i] * (p - 1);
      }
    }
    rep(i, 1, m){
      sphi[i] = sphi[i - 1] + phi[i];
    }
  }
  i64 solve(int n){
    if(n < int(size(phi))){
      return sphi[n];
    }
    if(auto it = mem.find(n); it != mem.end()){
      return it->se;
    }
    i64 res = i64(n) * (i64(n) + 1) / 2;
    int s = 2;
    for(; i64(s) * s <= n; s++){
      int k = n / s;
      res -= solve(k) + i64(phi[s]) * k;
    }
    s--;
    res += i64(s) * solve(s) - n;
    return mem[n] = res;
  }
};
```

Min_25 筛

number_theory/min25.cpp

```cpp
// Min_25 示例：积性 f，f(p^e)=p^e*(p^e-1)，f(1)=1。
// solve(n) 返回 sum_{i=1}^n f(i) mod 1e9+7；0<=n<=1e10，solve(0)=0。
// 先筛质数的一次、二次幂和，再按最小质因子枚举质数幂；普通整数除法。
struct Min25{
  static const int mod = 1000000007;
  i64 n = 0;
  int sq = 0, m = 0;
  vi pr, f, g;
  // 仅用于当前 n 的不同正整除商，编号 [1,m]，值递增。
  int id(i64 x){
    return (x <= sq ? int(x) : m + 1 - int(n / x));
  }
  i64 val(int i){
    return (i <= sq ? i64(i) : n / (m + 1 - i));
  }
  // 排除前 i 个质因子，求 2..x 的 f 之和；x 须为当前 n 的正整除商。
  int calc(i64 x, int i){
    if(x <= 1 || (i && x <= pr[i - 1])){
      return 0;
    }
    int j = id(x);
    i64 res = g[j] - f[j];
    if(i){
      res -= g[pr[i - 1]] - f[pr[i - 1]];
    }
    for(int k = i; k < int(size(pr)) && i64(pr[k]) * pr[k] <= x; k++){
      i64 p = pr[k];
      for(i64 pw = p; pw <= x / p; pw *= p){
        i64 a = pw % mod, b = pw * p % mod;
        res = (res + a * (a - 1) % mod * calc(x / pw, k + 1) + b * (b - 1)) % mod;
      }
    }
    return int(res % mod);
  }
  int solve(i64 _n){
    n = _n;
    sq = int(sqrt(n));
    while(i64(sq + 1) * (sq + 1) <= n){
      sq++;
    }
    while(i64(sq) * sq > n){
      sq--;
    }
    m = sq + int(n / (sq + 1));
    pr.clear();
    vi np(sq + 1);
    rep(i, 2, sq){
      if(!np[i]){
        pr.pb(i);
        for(i64 j = i64(i) * i; j <= sq; j += i){
          np[int(j)] = 1;
        }
      }
    }
    f.assign(m + 1, 0), g.assign(m + 1, 0);
    rep(i, 1, m){
      i64 x = val(i) % mod;
      f[i] = int(x * (x + 1) / 2 % mod - 1);
      g[i] = int(x * (x + 1) % mod * (2 * x + 1) % mod * 166666668 % mod - 1);
    }
    for(int p:pr){
      per(i, m, 1){
        i64 x = val(i);
        if(x < i64(p) * p){
          break;
        }
        int j = id(x / p);
        f[i] = int((f[i] - i64(p) * (f[j] - f[p - 1])) % mod);
        g[i] = int((g[i] - i64(p) * p % mod * (g[j] - g[p - 1])) % mod);
      }
    }
    return (n ? (calc(n, 0) + 1 + mod) % mod : 0);
  }
};
```

PN 筛

number_theory/pn_sieve.cpp

```cpp
// PN 筛：F=G*H，F、G 积性，F(p)=G(p)，F(1)=G(1)=1。
// 返回 sum_{i=1}^n F(i) mod mod；0<=n<=2^62-1，1<=mod<=INT_MAX。
// pr 递增且不重复，包含所有 <=sqrt(n) 的质数；不修改、不保存输入引用。
// h(p,e,pw)=H(p^e)，e>=2、pw=p^e；sum_g(x)=sum_{i<=x}G(i)。
// 回调均返回 [0,mod)；仅枚举 PN，时间另加回调及质数表预处理开销。
int pn_sieve(i64 n, const vi &pr, int mod, auto h, auto sum_g){
  auto dfs = [&](auto &self, i64 x, int i, i64 v)->int {
    i64 res = v * sum_g(x) % mod;
    for(int j = i; j < int(size(pr)) && i64(pr[j]) * pr[j] <= x; j++){
      i64 p = pr[j], pw = p * p;
      for(int e = 2; pw <= x; e++){
        res = (res + self(self, x / pw, j + 1, v * h(pr[j], e, pw) % mod)) % mod;
        if(pw > x / p){
          break;
        }
        pw *= p;
      }
    }
    return int(res);
  };
  return (n ? dfs(dfs, n, 0, 1 % mod) : 0);
}
```

PN 筛与杜教筛

number_theory/pn_dujiao.cpp

```cpp
// P5325：积性 F(p^e)=p^e*(p^e-1)，F(1)=1；先粘贴 pn_sieve.cpp。
// G(n)=n*phi(n)，F=G*H，H(p^e)=(e-1)*p^e*(p-1)。
// (n*phi)*id=id^2，用杜教筛求 G 前缀，再枚举 PN 合并。
```

```
// solve(n)：0<=n<=1e10，返回 [1,n] 的 F 和 mod 1e9+7；solve(0)=0。
// 预筛 [1,m]，1<=m<INT_MAX，所有查询须 floor(sqrt(n))<=m。
struct PNDujiao{
  static const int mod = 1000000007;
  vi pr, phi, sg;
  unordered_map<i64, int> mem;
  PNDujiao(int m = 5000000): phi(m + 1), sg(m + 1){
    phi[1] = 1;
    rep(i, 1, m){
      if(!phi[i]){
        pr.pb(i);
        phi[i] = i - 1;
      }
      for(int p:pr){
        if(p > m / i){
          break;
        }
        int j = i * p;
        if(i % p == 0){
          phi[j] = phi[i] * p;
          break;
        }
        phi[j] = phi[i] * (p - 1);
      }
      sg[i] = int((sg[i - 1] + i64(i) * phi[i]) % mod);
    }
  }
  int sum_g(i64 n){
    if(n < i64(size(sg))){
      return sg[int(n)];
    }
    if(auto it = mem.find(n); it != mem.end()){
      return it->se;
    }
    i64 x = n % mod;
    i64 res = x * (x + 1) % mod * (2 * x + 1) % mod * 166666668 % mod
      - x * (x + 1) / 2 % mod;
    int s = 2;
    for(; i64(s) * s <= n; s++){
      i64 k = n / s, t = k % mod;
      res = (res - i64(s) * sum_g(k)
        - i64(s) * phi[s] % mod * (t * (t + 1) / 2 % mod)) % mod;
    }
    s--;
    res = (res + i64(s) * (s + 1) / 2 % mod * sum_g(s)) % mod;
    if(res < 0){
      res += mod;
    }
    return mem[n] = int(res);
  }
  int solve(i64 n){
    auto h = [&](int p, int e, i64 pw){
      return int(i64(e - 1) * (pw % mod) % mod * (p - 1) % mod);
    };
    return pn_sieve(n, pr, mod, h, [&](i64 x){return sum_g(x);});
  }
};
```

倒数乘法除法

number_theory/qdiv.cpp

```
// 倒数乘法除法，仅作常数优化参考；不自动替换其他模板中的除法。
// 先 init()，2<=y<B；0<=x<2^48 对当前 B 保证正确，y=1 不支持。
// 一般充分条件：x<2^64 且 x*(y-1)<2^64（按数学整数计算）。
// u128 参数用于扩展乘法，不表示支持任意 128 位被除数。
namespace QDiv{
  using u64 = unsigned long long;
  using u128 = unsigned __int128;
  const int B = 65536 + 114;
  u64 inv[B];
  void init(){
    rep(i, 1, B - 1){
      inv[i] = (-1ULL) / u64(i) + 1;
    }
  }
  u64 qdiv(u128 x, u64 y){
    return u64(x * inv[y] >> 64);
  }
}
```

形式幂级数与多项式

mint_poly.cpp

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define rep(i, j, k) for(int i = (j); i <= (k); i++)
#define per(i, j, k) for(int i = (j); i >= (k); i--)
#define pb emplace_back
#define fi first
#define se second
using vi = vector<int>;
using pi = pair<int, int>;

bool chmin(auto &x, const auto &y){
  if(y < x){x = y; return true;} return false;
}
bool chmax(auto &x, const auto &y){
  if(x < y){x = y; return true;} return false;
}

ostream& operator << (ostream &os, const auto &v)
  requires requires { v.begin(); } && (!requires { v.substr(); }) {
  os << "[";
  int f = 0;
  for (const auto& x : v) os << (f++ ? ", " : "") << x;
  return os << "]";
}

template<typename ...Ar> void debug(char *s, Ar... x){
  auto f = [&](auto x){
    int dep = 0;
    while(*s && !(*s == ',' && dep == 0)){
      if(*s == '(') dep++;
      if(*s == ')') dep--;
      cerr << *s++;
    }
    cerr <<" = "<< x;
    if(*s == ','){
      cerr << *s ++;
    } else{
      cerr << endl;
    }
  };

  (f(x), ...);
}
#define gdb(...) debug((char*)#__VA_ARGS__, __VA_ARGS__)
```

模整数运算

```
using u32 = uint32_t;
using u64 = uint64_t;
constexpr int mod = 998244353;
struct mint{
  u32 x;

  mint(): x(0){}
  mint(int _x){
    _x %= mod;
    if(_x < 0) _x += mod;
    x = _x;
  }

  mint qpow(int y = mod - 2)const {
    assert(y >= 0);
    mint t = *this, res = 1;
    while(y){
      if(y % 2) res *= t;
      t *= t;
      y /= 2;
    }
    return res;
  }

  mint& operator += (const mint &B){
    if((x += B.x) >= mod) x -= mod;
    return *this;
  }
  mint& operator -= (const mint &B){
    if((x -= B.x) >= mod) x += mod;
    return *this;
  }
  mint& operator *= (const mint &B){
    x = (u64)x * B.x % mod;
    return *this;
  }
  mint& operator /= (const mint &B){
    return *this *= B.qpow();
```

```
  }
  friend mint operator + (mint A, const mint &B){
    return mint(A) += B;
  }
  friend mint operator - (mint A, const mint &B){
    return mint(A) -= B;
  }
  friend mint operator * (mint A, const mint &B){
    return mint(A) *= B;
  }
  friend mint operator / (mint A, const mint &B){
    return mint(A) /= B;
  }
  mint operator - ()const {
    return mint() - *this;
  }

  friend ostream& operator << (ostream &os, const mint &obj){
    os << obj.x;
    return os;
  }
};

constexpr int LG = 21, S = 1 << LG;
vector<mint> P, fac, ifac, iv, ip2;
```

单位根、阶乘与逆元预处理

```
auto INIT = [](){
  P.resize(S);
  rep(i, 0, LG - 1){
    mint stp = mint(3).qpow(mod >> (i + 1));
    P[1 << i] = 1;
    rep(j, (1 << i) + 1, (2 << i) - 1){
      P[j] = P[j - 1] * stp;
    }
  }

  fac.resize(S);
  ifac.resize(S);
  iv.resize(S);
  fac[0] = iv[0] = 1;
  rep(i, 1, S - 1){
    fac[i] = fac[i - 1] * i;
  }
  ifac[S - 1] = 1 / fac[S - 1];
  per(i, S - 1, 1){
    ifac[i - 1] = ifac[i] * i;
    iv[i] = ifac[i] * fac[i - 1];
  }

  ip2.resize(LG + 1);
  ip2[0] = 1;
  rep(i, 1, LG){
    ip2[i] = ip2[i - 1] / 2;
  }

  return true;
}();

struct poly : vector<mint>{
  using vector<mint>::vector;
  using polyY = vector<poly>;

```

NTT : DIF 正变换

```
  void DIF(int lg){
    for(int i = 1 << (lg - 1); i ; i /= 2){
      for(int j = 0; j < (1 << lg); j += 2 * i){
        rep(k, 0, i - 1){
          mint A = (*this)[j + k], B = (*this)[j + k + i];
          (*this)[j + k] = A + B, (*this)[j + k + i] = (A - B) * P[i + k];
        }
      }
    }
  }

```

NTT : DIT 逆变换

```
  void DIT(int lg){
    for(int i = 1; i < (1 << lg); i *= 2){
      for(int j = 0; j < (1 << lg); j += 2 * i){
        rep(k, 0, i - 1){
          mint A = (*this)[j + k], B = (*this)[j + k + i] * P[i + k];
          (*this)[j + k] = A + B, (*this)[j + k + i] = A - B;
        }
      }
    }
    reverse(this -> begin() + 1, this -> end());
  }

```

多项式求导

```
  friend poly der(const poly &A){
    poly res = A;
    rep(i, 1, (int)res.size() - 1){
      res[i - 1] = res[i] * i;
    }
    res.pop_back();
    return res;
  }

```

多项式积分

```
  friend poly ints(const poly &A){
    poly res = A;
    res.pb();
    per(i, (int)res.size() - 1, 0){
      res[i] = res[i - 1] * iv[i];
    }
    res[0] = 0;
    return res;
  }

```

多项式四则运算与截断

```
  poly& operator += (const poly &B){
    if(B.size() > this -> size()){
      this -> resize(B.size());
    }
    rep(i, 0, (int)B.size() - 1){
      (*this)[i] += B[i];
    }
    return *this;
  }

  friend poly operator + (const poly &A, const poly &B){
    return poly(A) += B;
  }

  poly& operator -= (const poly &B){
    if(B.size() > this -> size()){
      this -> resize(B.size());
    }
    rep(i, 0, (int)B.size() - 1){
      (*this)[i] -= B[i];
    }
    return *this;
  }

  friend poly operator - (const poly &A, const poly &B){
    return poly(A) -= B;
  }

  friend poly operator * (const mint &k, const poly &A){
    poly res = A;
    for(auto &x:res) x *= k;
    return res;
  }

  poly& operator *= (const poly &B){
    auto sz = this -> size();
    *this = conv(*this, B);
    this -> resize(sz);
    return *this;
  }

  friend poly operator * (const poly &A, const poly &B){
    return poly(A) *= B;
  }

  poly& operator /= (const poly &B){
    auto sz = this -> size();
    *this = conv(*this, inv(B));
    this -> resize(sz);
    return *this;
  }

  friend poly operator / (const poly &A, const poly &B){
    return poly(A) /= B;
  }

```

多项式卷积

```
  friend poly conv(const poly &_A, const poly &_B){
    if(_A.size() <= 64 || _B.size() <= 64){
      poly res(_A.size() + _B.size() - 1);
      rep(i, 0, (int)_A.size() - 1){
        rep(j, 0, (int)_B.size() - 1){
          res[i + j] += _A[i] * _B[j];
        }
      }
      return res;
    }

    poly A = _A, B = _B;
    int len = A.size() + B.size() - 1;
    int lg = __lg(len * 2 - 1);
    A.resize(1 << lg), B.resize(1 << lg);
    A.DIF(lg), B.DIF(lg);
    rep(i, 0, (1 << lg) - 1){
      A[i] *= B[i];
    }
    A.DIT(lg);
    A.resize(len);
    for(auto &x:A) x *= ip2[lg];
    return A;
  }

```

多项式求逆

```
  friend poly inv(const poly &A){
    assert(A[0].x != 0);
    poly res{mint(1) / A[0]};
    while(res.size() < A.size()){
      int n = res.size(), lg = __lg(n * 4);
      poly buf(1 << lg);
      copy(res.begin(), res.end(), buf.begin());
      buf.DIF(lg);

      poly tmp{A.begin(), A.begin() + min(n * 2,
 > (int)A.size())};
      tmp.resize(n * 4);
      tmp.DIF(lg);
      rep(i, 0, n * 4 - 1) tmp[i] *= buf[i] * buf[i];
      tmp.DIT(lg);

      res.resize(n * 2);
      rep(i, 0, n * 2 - 1){
        res[i] = 2 * res[i] - tmp[i] * ip2[lg];
      }
    }
    res.resize(A.size());
    return res;
  }


```

多项式指数

```
  // O(n \log^2 n)
  friend poly exp(const poly &A){
    assert(A[0].x == 0);
    int n = A.size();
    int lg = __lg(n * 2 - 1);

    poly a = A, res(A.size());
    rep(i, 0, (int)a.size() - 1) a[i] *= i;
    a.resize(1 << lg);

    vector<poly> pre(lg + 1);
    rep(i, 8, lg){
      pre[i].insert(pre[i].end(), a.begin(), a.begin() +
 > (1 << i));
      pre[i].DIF(i);
    }

    res[0] = 1;
    auto slv = [&](auto &self, int l, int r)->void {
      if(r - l <= 128){
        rep(i, l, r - 1){
          rep(j, l, i - 1){
            res[i] += res[j] * a[i - j];
          }
          if(i > 0) res[i] *= iv[i];
        }
        return;
      }
      int mid = (l + r) / 2;
      self(self, l, mid);

      int _lg = __lg((r - l) * 2 - 1);
      poly tmp(res.begin() + l, res.begin() + mid);
      tmp.resize(1 << _lg);
      tmp.DIF(_lg);
      rep(i, 0, (1 << _lg) - 1) tmp[i] *= pre[_lg][i];
      tmp.DIT(_lg);
      rep(i, mid, r - 1) res[i] += tmp[i - l] * ip2[_lg];
      self(self, mid, r);
    };
    slv(slv, 0, n);
    return res;
  }

```

多项式对数

```
  friend poly ln(const poly &A){
    assert(A[0].x == 1);
    poly res = der(A) / A;
    return ints(res);
  }

```

多项式平方根

```
  friend poly sqrt(const poly &A){
    assert(A[0].x == 1);
    return exp(ip2[1] * ln(A));
  }

```

多项式幂

```
  friend poly qpow(const poly &A, mint k){
    if(k.x == 0){
      poly res(A.size());
      res[0] = 1;
      return res;
    }

    auto it = A.begin();
    while(it != A.end() && it -> x == 0){
      it ++;
    }
    int len = (it - A.begin()) * k.x;
    if(len >= (int)A.size()){
      return poly(A.size());
    } else{
      poly a{it, it + (A.size() - len)};
      mint t = a[0].qpow(k.x);
      a = t * exp(k * ln(1 / a[0] * a));
      a.insert(a.begin(), len, mint());
      return a;
    }
  }

```

多项式除法与余数

```
  friend pair<poly, poly> division(const poly &_P, const
 > poly &_Q){
    auto lenF = _P.size() - _Q.size() + 1;
    auto lenR = _Q.size() - 1;
    poly rP(_P.rbegin(), _P.rbegin() + lenF);
    poly rQ(_Q.rbegin(), _Q.rbegin() + min(_Q.size(),
 > lenF));
    rQ.resize(lenF);
    poly F = rP / rQ;
    reverse(F.begin(), F.end());
    poly R = _P - conv(_Q, F);
    R.resize(lenR);
    return {F, R};
  }

```

转置卷积

```
  friend poly convT(const poly &_A, const poly &_B){
    poly A = _A, B = _B;
    int k = B.size() - 1;
    reverse(B.begin(), B.end());
    int lg = __lg(A.size() * 2 - 1);
    A.resize(1 << lg), B.resize(1 << lg);
    A.DIF(lg), B.DIF(lg);
    rep(i, 0, (1 << lg) - 1){
      A[i] *= B[i];
    }
    A.DIT(lg);
    A = poly{A.begin() + k, A.begin() + _A.size()};
    for(auto &x:A){
      x *= ip2[lg];
    }
    return A;
```

```
  }
```

多点求值

```
  friend poly multipoint_eval(poly f, const poly &x){
    vector<poly> Q(x.size() * 2 - 1);
    poly res;

    auto dfs0 = [&](auto &self, int u, int l, int r)->void {
      if(l + 1 == r){
        Q[u] = {1, -x[l]};
        return;
      }
      int mid = (l + r) / 2, ls = u + 1, rs = u + (mid - l) * 2;
      self(self, ls, l, mid);
      self(self, rs, mid, r);
      Q[u] = conv(Q[ls], Q[rs]);
    };
    dfs0(dfs0, 0, 0, x.size());

    auto dfs1 = [&](auto &self, int u, int l, int r, poly F)->void {
      F.resize(r - l);
      if(l + 1 == r){
        res.pb(F[0]);
        return;
      }
      int mid = (l + r) / 2;
      int ls = u + 1, rs = u + (mid - l) * 2;
      auto FL = convT(F, Q[rs]), FR = convT(F, Q[ls]);
      self(self, ls, l, mid, FL);
      self(self, rs, mid, r, FR);
    };
    Q[0].resize(f.size());
    poly g = inv(Q[0]);
    g.resize(f.size());
    f.resize(f.size() * 2 - 1);
    g = convT(f, g);
    dfs1(dfs1, 0, 0, x.size(), g);

    return res;
  }
```

二维卷积

```
  friend polyY conv(const polyY &A, const polyY &B){
    int x = A.size() + B.size() - 1;
    int y = A[0].size() + B[0].size() - 1;
    poly tmpa(A.size() * y), tmpb(B.size() * y);
    rep(i, 0, int(A.size()) - 1){
      rep(j, 0, int(A[i].size()) - 1){
        tmpa[i * y + j] = A[i][j];
      }
    }
    rep(i, 0, int(B.size()) - 1){
      rep(j, 0, int(B[i].size()) - 1){
        tmpb[i * y + j] = B[i][j];
      }
    }
    poly tmpc = conv(tmpa, tmpb);
    polyY res(x, poly(y));
    rep(i, 0, x - 1){
      rep(j, 0, y - 1){
        res[i][j] = tmpc[i * y + j];
      }
    }
    return res;
  }
```

二维转置卷积

```
  friend polyY convT(const polyY &A, const polyY &B){
    int n = A.size(), m = A[0].size(), k = B[0].size() - 1;
    poly a(n * m), b(B.size() * m);
    rep(i, 0, n - 1){
      rep(j, 0, m - 1){
        a[i * m + j] = A[i][j];
      }
    }
    rep(i, 0, int(B.size()) - 1){
      rep(j, 0, int(B[i].size()) - 1){
        b[i * m + j] = B[B.size() - 1 - i][k - j];
      }
    }
    int lg = __lg(a.size() * 2 - 1);
    a.resize(1 << lg), b.resize(1 << lg);
    a.DIF(lg), b.DIF(lg);
    rep(i, 0, (1 << lg) - 1){
      a[i] *= b[i];
    }
    a.DIT(lg);

    polyY res(A.size() - B.size() + 1, poly(A[0].size() - B[0].size() + 1));
    rep(i, 0, int(res.size()) - 1){
      rep(j, 0, int(res[i].size()) - 1){
        res[i][j] = a[(i + B.size() - 1) * m + j + k] * ip2[lg];
      }
    }
    return res;
  }
```

Bostan-Mori

```
  friend poly BostanMori(int n, polyY P, polyY Q){
    if(n == 0){
      return P[0] * inv(Q[0]);
    }
    polyY tQ = Q;
    for(int i = 1; i < int(tQ.size()); i += 2){
      tQ[i] = poly{} - tQ[i];
    }
    polyY nxtP = conv(P, tQ), nxtQ = conv(Q, tQ);
    P = polyY(n / 2 + 1, poly(nxtP[0].size())), Q = P;
    rep(i, 0, int(P.size()) - 1){
      P[i] = nxtP[i * 2 + n % 2];
      Q[i] = nxtQ[i * 2];
    }
    return BostanMori(n / 2, P, Q);
  }
```

转置 Bostan-Mori

```
  friend polyY BostanMoriT(int n, poly P, polyY Q){
    if(n == 0){
      P.resize(Q[0].size() * 2 - 1);
      return {convT(P, inv(Q[0]))};
    }
    polyY tQ = Q;
    for(int i = 1; i < int(tQ.size()); i += 2){
      tQ[i] = poly{} - tQ[i];
    }
    polyY rQ = conv(Q, tQ);
    polyY nxtQ(n / 2 + 1, poly(rQ[0].size()));
    rep(i, 0, int(nxtQ.size()) - 1){
      nxtQ[i] = rQ[i * 2];
    }
    polyY nxtP = BostanMoriT(n / 2, P, nxtQ);
    polyY res(tQ.size() * 2 - 1, poly(nxtP[0].size()));
    rep(i, 0, int(nxtP.size()) - 1){
      res[i * 2 + n % 2] = nxtP[i];
    }
    return convT(res, tQ);
  }
```

多项式复合逆

```
  friend poly composite_inverse(const poly &_F){
    int n = _F.size();
    mint v = 1 / _F[1];

    poly F = v * _F;
    polyY P, Q;
    rep(i, 0, n - 1){
      P.pb(poly{i == 0, 0});
      Q.pb(poly{i == 0, -F[i]});
    }
    poly res = BostanMori(n, P, Q);
    poly tmp(n - 1);
    rep(k, 2, n){
      mint val = res[k];
      val = val * n * iv[k];
      tmp[n - k] = val;
    }

    poly ans = qpow(tmp, -mint(1) / n);
    ans.insert(ans.begin(), 0);
    ans.resize(n);

    mint pw = 1;
    rep(i, 1, int(ans.size()) - 1){
      pw *= v;
      ans[i] *= pw;
    }
```

```
    return ans;
  }
```

多项式复合

```
  // G(F(x))
  friend poly composite(const poly &G, const poly &F){
    poly P = G;
    polyY Q;
    rep(i, 0, int(G.size()) - 1){
      Q.pb(poly{(i == 0), (i < int(F.size())? -F[i]: 0)});
    }

    auto res = BostanMoriT(G.size() - 1, P, Q);
    poly ans(res.size());
    rep(i, 0, int(ans.size()) - 1){
      ans[i] = res[ans.size() - 1 - i][0];
    }
    return ans;
  }
};
```

在线卷积

```
struct dc{
  poly f, g, res;
  vector<poly> pf, pg;
  int n, p;

  dc(int _n) : f(_n), g(_n), res(_n), n(_n), p(0) {}

  void push(mint v0, mint v1){
    f[p] = v0;
    res[p] += f[0] * v1 + g[0] * v0;
    g[p] = v1;
    p ++;

    int t = __builtin_ctz(p), len = 1 << t;
    poly A(f.begin() + p - len, f.begin() + p), B(g.begin() + p - len, g.begin() + p);
    A.resize(len * 2), B.resize(len * 2);
    A.DIF(t + 1), B.DIF(t + 1);
    if(len == p){
      if(len > 1){
        poly C(f.begin(), f.begin() + p), D(g.begin(), g.begin() + p);
        C.DIF(t), D.DIF(t);
        pf.pb(C), pg.pb(D);
      }
      rep(i, 0, len * 2 - 1){
        A[i] *= B[i];
      }
    } else{
      auto &C = pf[t], &D = pg[t];
      rep(i, 0, len * 2 - 1){
        A[i] = (A[i] * D[i] + B[i] * C[i]);
      }
    }
    A.DIT(t + 1);
    rep(i, p, min(p + len, n) - 1){
      res[i] += A[i - (p - len)] * ip2[t + 1];
    }
  }
};
```

在线多项式对数

```
struct dln : dc{
  poly res;
  bool bg;

  dln(int _n): dc(_n), res(_n), bg(0) {}

  void push(mint v){
    if(!bg){
      bg = 1;
    } else{
      res[p + 1] = v - (p == 0? 0: dc::res[p - 1] * iv[p + 1]);
      dc::push((p + 1) * res[p + 1], v);
    }
  }
};
```

在线多项式指数

```
struct dexp : dc{
  poly res;

  dexp(int _n): dc(_n), res(_n) {}

  void push(mint v){
    if(!res[0].x){
      res[0] = 1;
    } else{
      v *= (p + 1);
      dc::push(v, res[p]);
      res[p] = dc::res[p - 1] * iv[p];
    }
  }
};
```

在线多项式求逆

```
struct dinv : dc{
  poly res;
  mint bg;

  dinv(int _n): dc(_n), res(_n), bg(0) {}

  void push(mint v){
    if(!bg.x){
      bg = v, res[0] = 1 / v;
    } else{
      dc::push(v, res[p]);
      res[p] = -res[0] * dc::res[p - 1];
    }
  }
};
```

阶乘组合数

```
mint C(int m, int n){
  return n < 0 || n > m? 0: fac[m] * ifac[n] * ifac[m - n];
}
```

下降幂组合数

```
mint binom(int m, int n){
  mint res = ifac[n];
  rep(i, 0, n - 1){
    res *= m - i;
  }
  return res;
}

signed main(){
  #ifdef LOCAL
  assert( freopen(".in", "r", stdin) );
  assert( freopen(".out", "w", stdout) );
  #endif
  ios::sync_with_stdio(0);
  cin.tie(0);

  poly a{1, 2, 3};
  cout << a <<'\n';
}
```